Insper

ENGENHARIA

CYBERSEGURANÇA EM NUVEM

Roteiro 2 – Controle de acesso adm e regras automatizadas

Eduardo Machado de Oliveira

Gustavo Eizo Takahashi

Henrique Leite dos Santos

Lucas Kenji Kamikawa

Sidney Barbosa de Oliveira

Prof. Rodolfo Avelino

São Paulo

07/10/2026

Sumário

Tarefa 1 – Acesso administrativo seguro com Apache Guacamole 3
1.1 Fundamentação Teórica 3
1.2 Instalação e Exposição Inicial 3
1.3 Hardening do Guacamole 5
1.4 Fechamento do Acesso Direto nos Servidores 7
Tarefa 2 – Detecção e resposta automatizada utilizando Cloudflare 9
2.1 Detecção e Definição de Parâmetros 10
2.2 Resposta Automatizada na Cloudflare 12
Tarefa 3 – Restringindo o acesso ao servidor Web 15
3.1 Configuração da Prefix List 15
3.2 Validação da Restrição 17

Tarefa 1 – Acesso administrativo seguro com Apache Guacamole

1.1 Fundamentação Teórica

O papel de um Jump Server/Bastion Host e suas vantagens: Um Jump Server atua como um ponto único e seguro de entrada (gateway) para a rede privada ou servidores internos da organização. Sua principal vantagem é reduzir drasticamente a superfície de ataque: em vez de expor o serviço SSH ou RDP de múltiplos servidores para a Internet pública, expõe-se apenas o Jump Server. Isso centraliza a auditoria, o controle de acesso e o monitoramento.

Riscos decorrentes da centralização e medidas de mitigação: A centralização cria um Ponto Único de Falha (SPOF) e um alvo de altíssimo valor (se comprometido, concede acesso a toda a infraestrutura). Para reduzir esses riscos, adotamos as seguintes medidas de *hardening* no nosso Apache Guacamole:

- **Exposição mínima:** O único serviço exposto à Internet é o portal web via HTTPS (porta 443). O tráfego SSH interno passa a ser restrito à rede privada/local.
- **Restrição de Origem (Allowlist):** O painel não é público. O Nginx foi configurado para aceitar tráfego apenas dos endereços IP conhecidos dos administradores do grupo.
- **Autenticação Forte:** Exigência de MFA (Múltiplo Fator de Autenticação) através de TOTP obrigatório para todos os usuários.

1.2 Instalação e Exposição Inicial

Primeiramente, foi criada uma instância EC2 dedicada (Ubuntu) na VPC do grupo para atuar como o nosso Bastion Host.

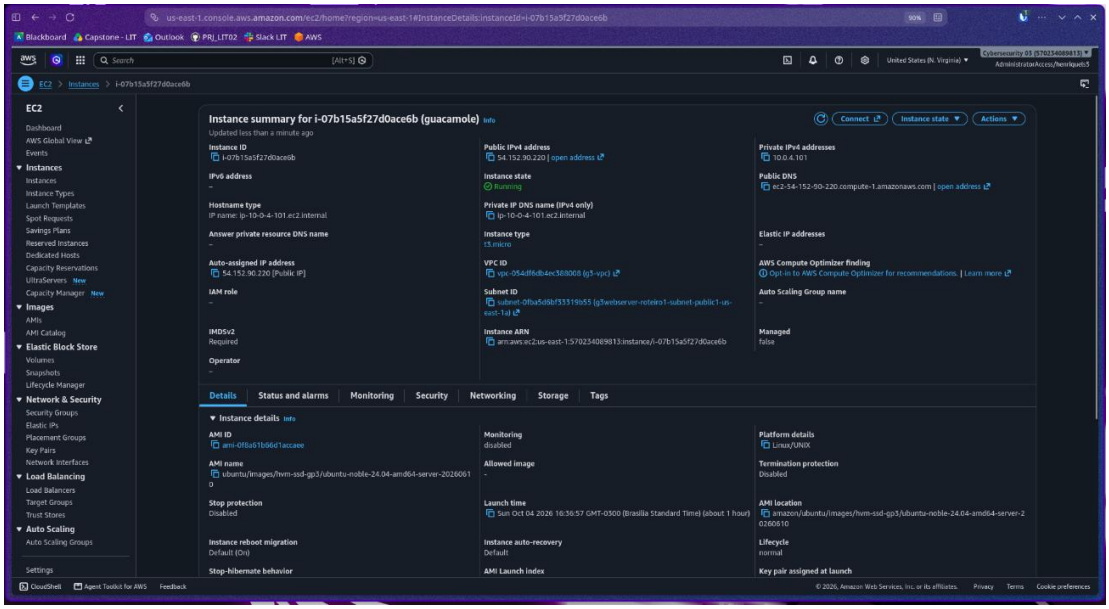
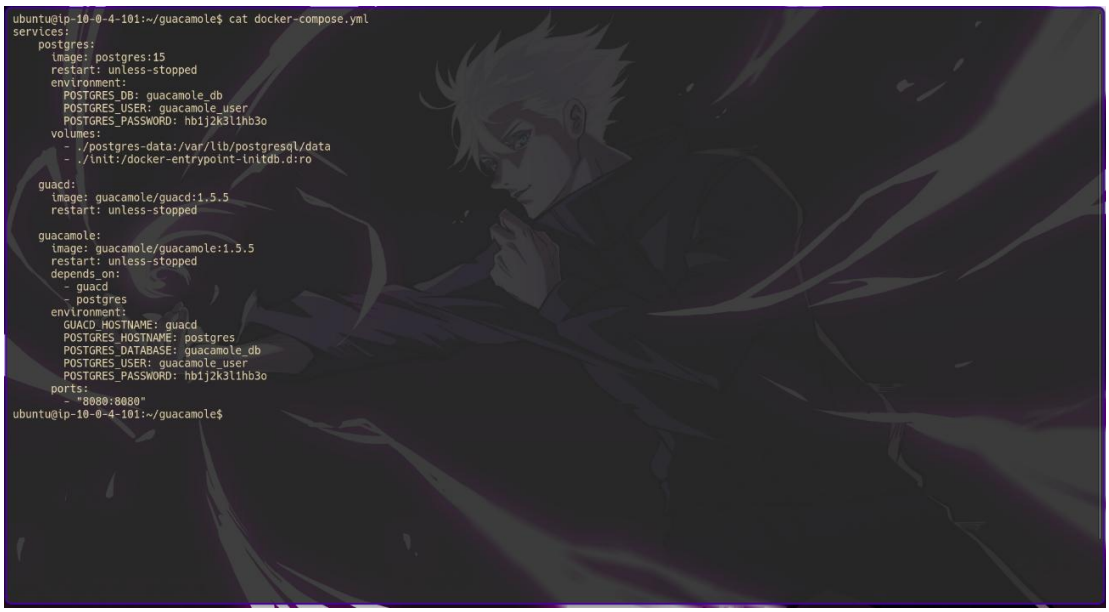


Figura 1 - Instância EC2 Guacamole em execução na AWS.

O Apache Guacamole foi instalado utilizando Docker Compose, subindo de forma conteinerizada a aplicação web, o daemon de proxy (guacd) e o banco de dados (postgres).



```
ubuntu@ip-10-0-4-101:~/guacamole$ cat docker-compose.yml
services:
    postgres:
      image: postgres:15
      restart: unless-stopped
      environment:
        POSTGRES_DB: guacamole_db
        POSTGRES_USER: guacamole_user
        POSTGRES_PASSWORD: hb1j2k3l1hb3o
      volumes:
        - ./postgres-data:/var/lib/postgresql/data
        - ./init:/docker-entrypoint-initdb.d:ro

    guacd:
      image: guacamole/guacd:1.5.5
      restart: unless-stopped

    guacamole:
      image: guacamole/guacamole:1.5.5
      restart: unless-stopped
      depends_on:
        - guacd
        - postgres
      environment:
        GUACD_HOSTNAME: guacd
        POSTGRES_HOSTNAME: postgres
        POSTGRES_DATABASE: guacamole_db
        POSTGRES_USER: guacamole_user
        POSTGRES_PASSWORD: hb1j2k3l1hb3o
      ports:
        - "8080:8080"
ubuntu@ip-10-0-4-101:~/guacamole$
```

Figura 2 - Subida e verificação dos serviços via Docker Compose.

Para garantir a exposição segura inicial, o Security Group da instância foi configurado para aceitar apenas requisições HTTPS oriundas da Cloudflare.

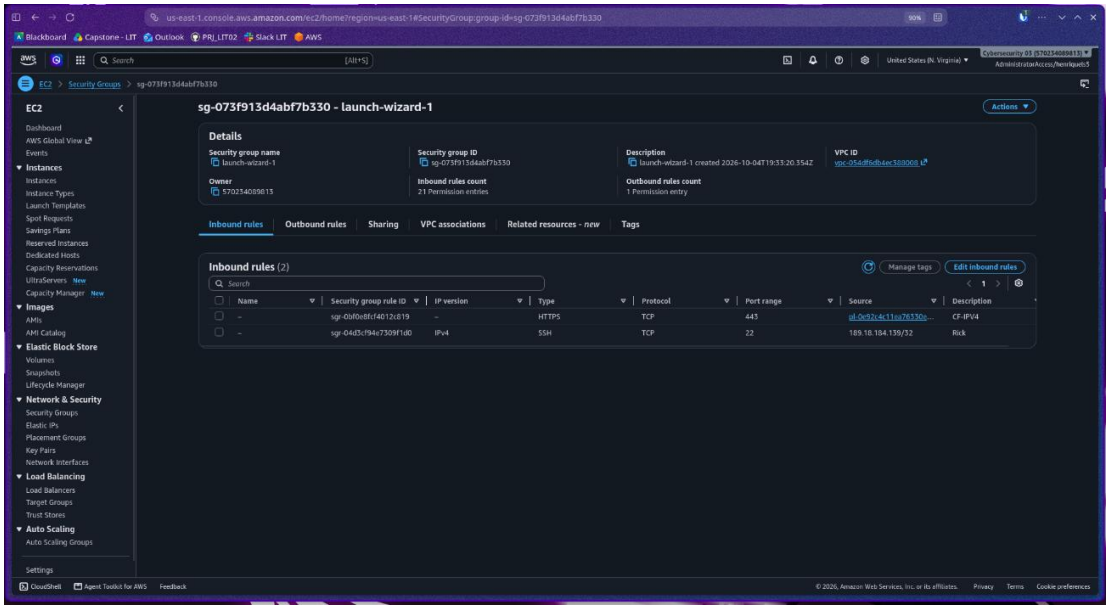


Figura 3 - Security Group do Guacamole aceitando apenas HTTPS (TCP/443).

Com o proxy reverso Nginx configurado, pudemos acessar a plataforma pela internet de forma segura via HTTPS.

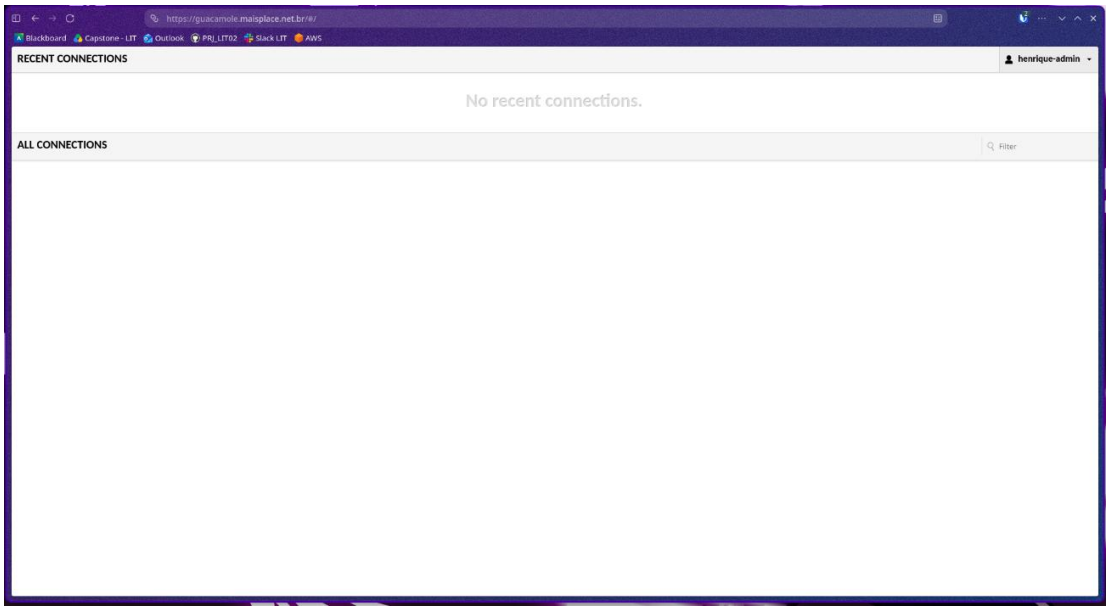


Figura 4 - Acesso à plataforma por HTTPS autenticado com usuário administrativo.

1.3 Hardening do Guacamole

Para fortalecer a segurança contra roubo de credenciais, o Segundo Fator de Autenticação (TOTP) foi ativado e tornado obrigatório.



Figura 5 - Segundo fator TOTP obrigatório ativado após a senha.

Em seguida, garantimos que nenhum serviço dos contêineres fosse exposto diretamente às interfaces públicas, restringindo a comunicação apenas ao localhost.

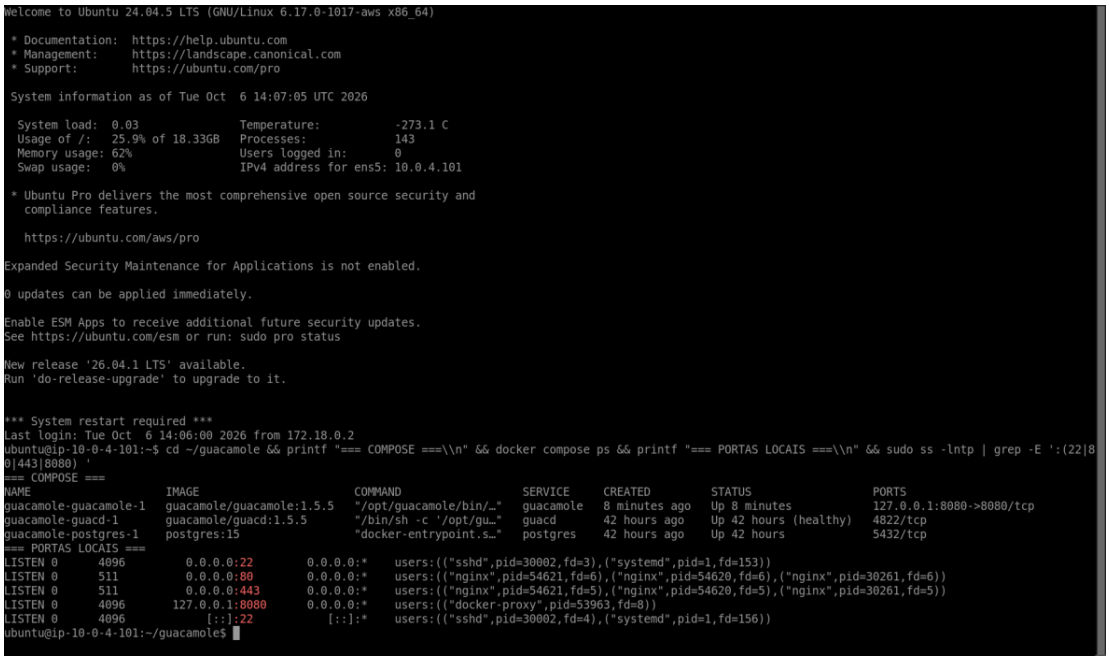


Figura 6 - Contêineres ativos e porta 8080 vinculada exclusivamente ao loopback.

Também criamos uma *Allowlist* rigorosa no Nginx. Apenas os IPs dos administradores (lidos corretamente a partir do cabeçalho da Cloudflare) podem abrir a página de login.



Figura 7 - Nginx Allowlist: IP autorizado recebe HTTP 200; IP fora da lista recebe HTTP 403.

Como camada final de proteção interna na máquina (Defesa em Profundidade), o *Uncomplicated Firewall* (UFW) foi ativado.

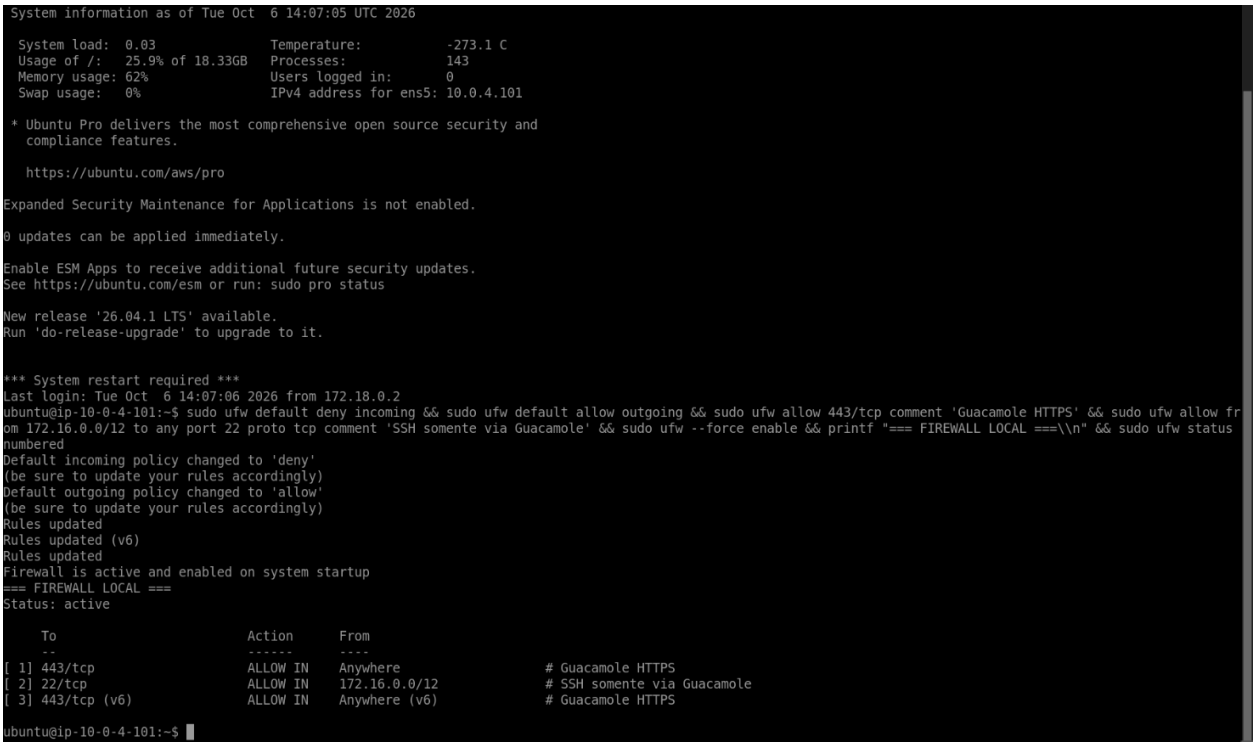


Figura 8 - UFW ativo no servidor Guacamole permitindo apenas regras essenciais.

1.4 Fechamento do Acesso Direto nos Servidores

Após a estabilização do Guacamole, criamos a conexão SSH administrativa apontando para o IP privado do nosso Webserver.

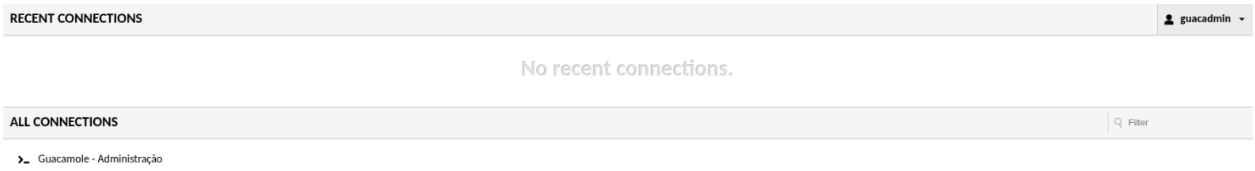


Figura 9 - Conexão SSH administrativa configurada na interface do Guacamole.

Realizamos a inicialização da sessão através do portal, sem a necessidade de expor chaves localmente.

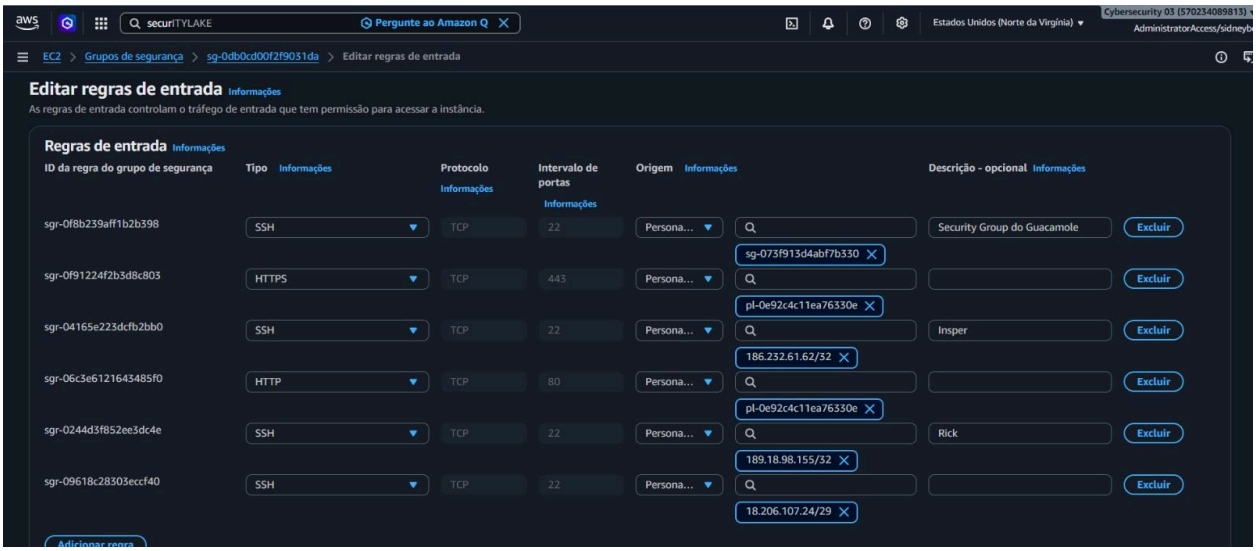


Figura 10 - Inicialização da conexão administrativa pelo painel web.

A conexão foi estabelecida com sucesso, abrindo o shell do Webserver.



Figura 11 - Sessão administrativa estabelecida no Webserver via Guacamole.

Para finalizar a Tarefa 1, fomos até a AWS e bloqueamos qualquer acesso SSH público direto ao Webserver, forçando o tráfego a passar exclusivamente pelo Guacamole.

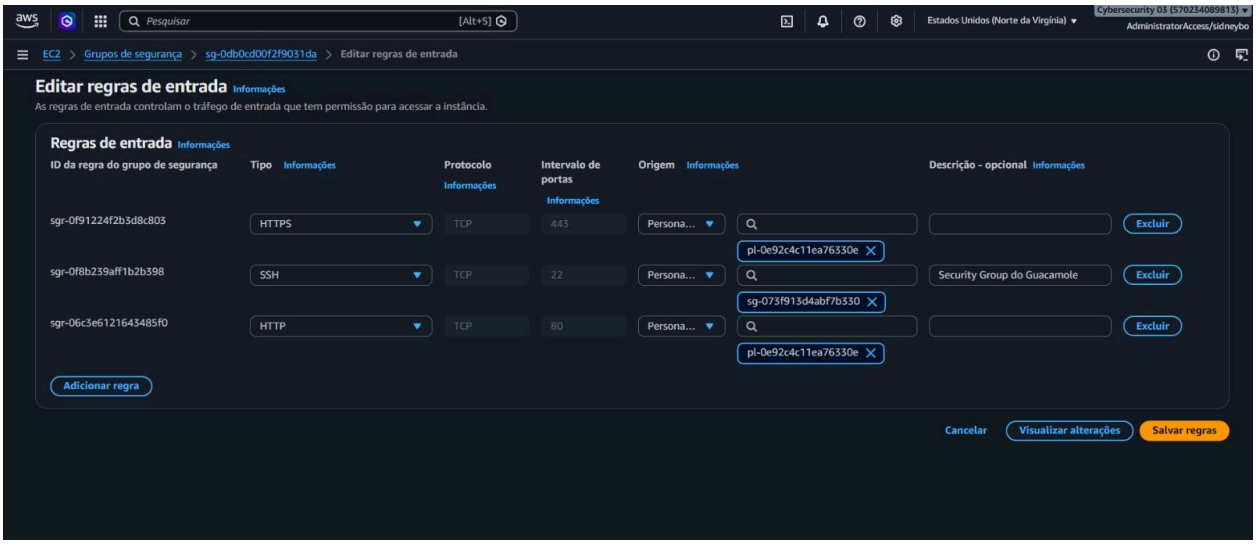


Figura 12 - Security Group do Webserver: Regra SSH restrita exclusivamente para a origem do Guacamole (sg-073f913d4abf7b330).

Tarefa 2 – Detecção e resposta automatizada utilizando Cloudflare

O objetivo desta tarefa foi automatizar o bloqueio de ataques de força bruta no /wp-login.php do WordPress. A automação completa seguiu as seguintes etapas: **Evento → Registro → Detecção → Decisão → Acionamento da API → Bloqueio → Validação**.

2.1 Detecção e Definição de Parâmetros

Para o *Registro* do ataque, o Apache foi configurado com o módulo mod_remoteip, extraindo o IP real do cliente a partir do cabeçalho CF-Connecting-IP repassado pela Cloudflare.



Figura 13 - Configuração do remoteip-cloudflare.conf gerado no Webserver garantindo o IP real.

Sem isso, todas as requisições pareceriam vir da própria Cloudflare, causando um banimento generalizado. O log foi alterado para gravar esse endereço real.

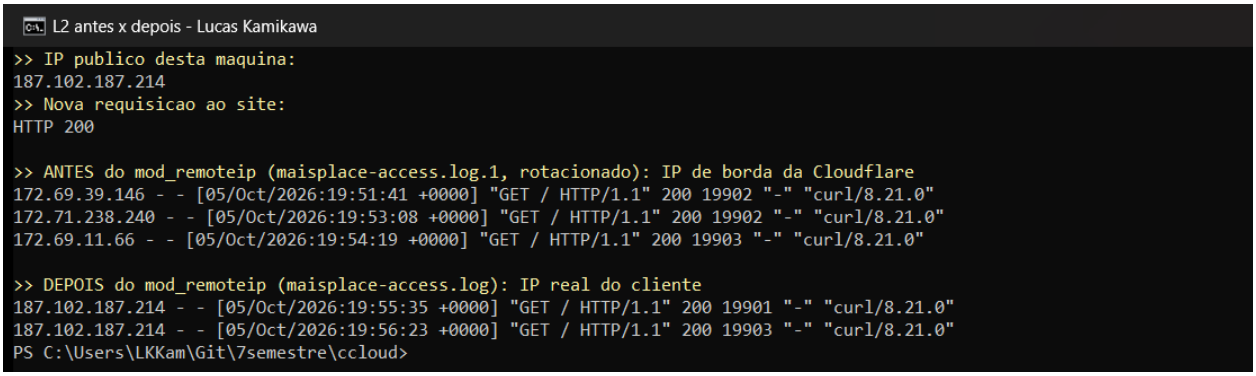


Figura 14 - maisplace-access.log registrando o IP correto do cliente.

Para a *Detecção* e *Decisão* pelo **Fail2ban**, definimos os seguintes parâmetros:

Parâmetro	Valor	Justificativa
Condição	POST em /wp-login.php com reposta 200	Falha de login no WP retorna 200 (sucesso retorna 302). Evita banir quem digita a senha certa.
Quantidade	5	Permite uma margem para usuários que esquecem a senha, mas pune robôs (Força

Parâmetro	Valor	Justificativa
		Bruta).
Intervalo	10 min	Pega ataques sequenciais automatizados sem penalizar usuários legítimos por falhas ao longo do dia.
Ação	1 hora	Tempo que o IP fica banido. Torna o ataque inviável, mas não prende permanentemente IPs compartilhados (CGNAT).
Identificação	CF-Connecting-IP	Graças ao mod_remoteip, o Fail2ban lê o log com a origem precisa do atacante, e não do proxy.

Iniciamos um Evento de ataque controlado, gerando diversas falhas de login de propósito a partir de um computador externo.

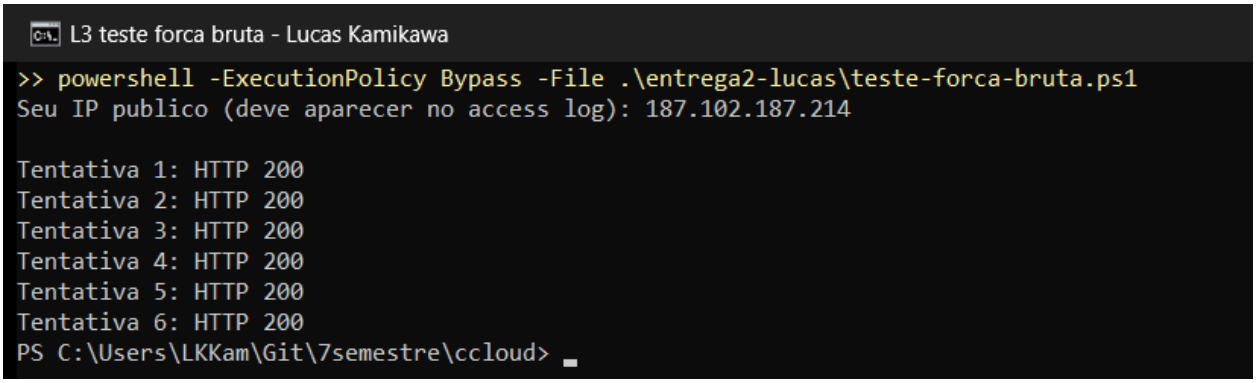


Figura 15 - Simulação de ataque com envio de 6 requisições POST com senha errada.

Essas falhas entraram perfeitamente nos logs do Apache e foram correspondidas (match) pelas Expressões Regulares do nosso filtro.

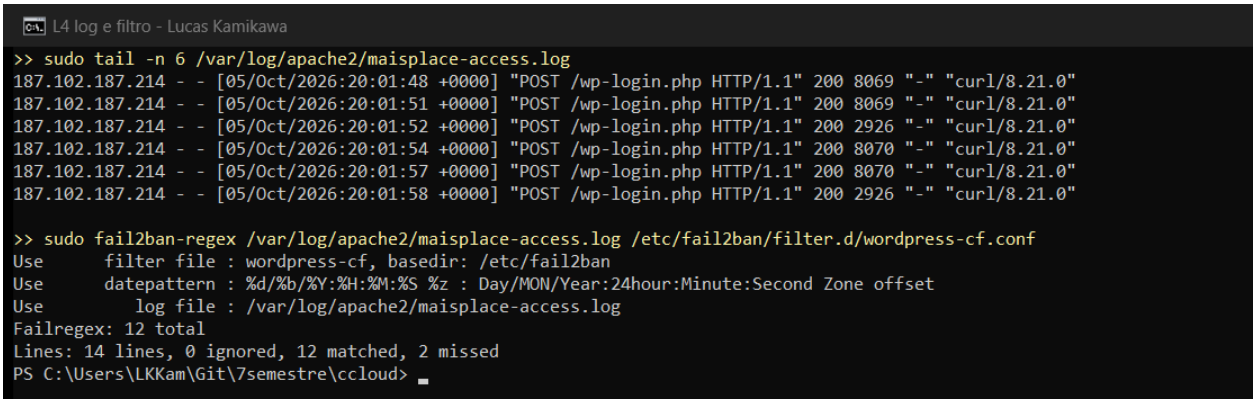


Figura 16 - Log do Webserver e Teste do Fail2ban-regex confirmando o reconhecimento do ataque.

Como configuramos o limite máximo para 5, na 6ª tentativa o Fail2ban detectou a anomalia e tomou a decisão de banir o IP.

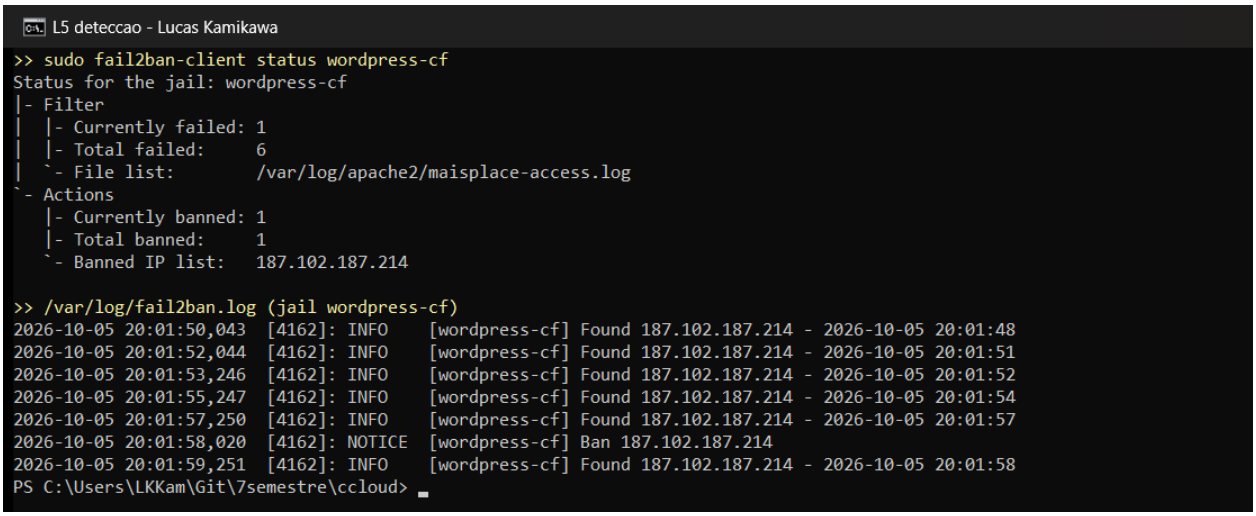


Figura 17 - Log interno do Fail2ban detectando as ocorrências e acionando o gatilho de "Ban".

2.2 Resposta Automatizada na Cloudflare

Com a decisão tomada localmente, precisamos do *Acionamento da API* para aplicar o bloqueio no WAF e proteger toda a infraestrutura. Geramos um Token na Cloudflare utilizando a premissa de Privilégio Mínimo (apenas para editar regras de Firewall do nosso domínio específico e apenas se requisitado pelo IP do servidor).

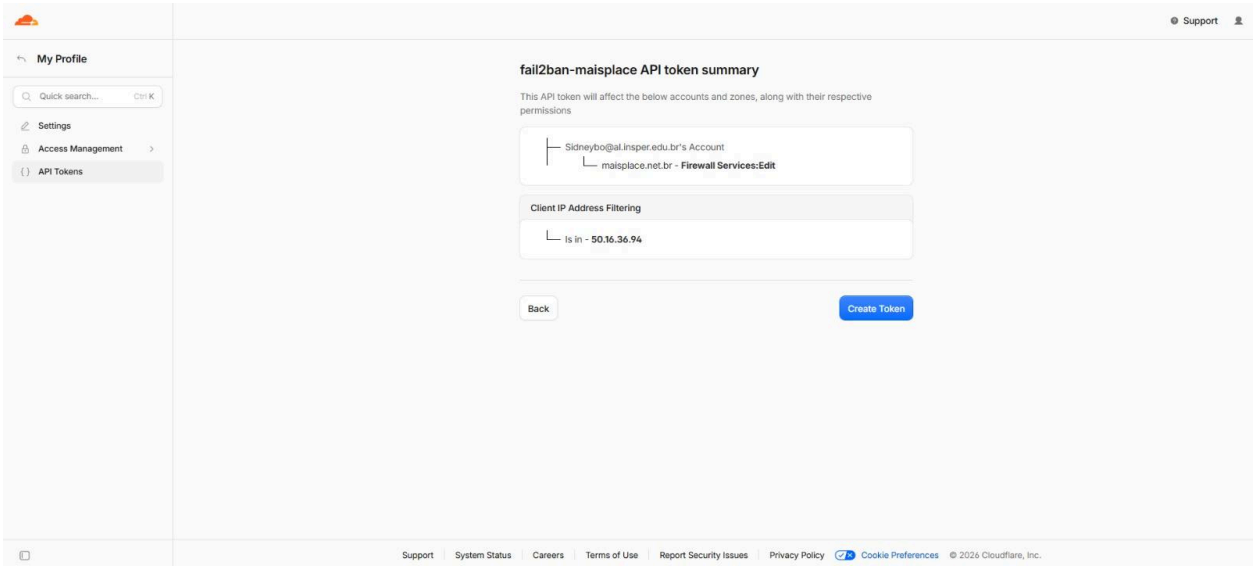


Figura 18 - Criação do Token na Cloudflare demonstrando as permissões mínimas e restrição de IP.

A credencial foi salva na máquina com permissões exclusivas do superusuário (chmod 600), e a Jail do Fail2ban foi ligada a uma Action local de bash.

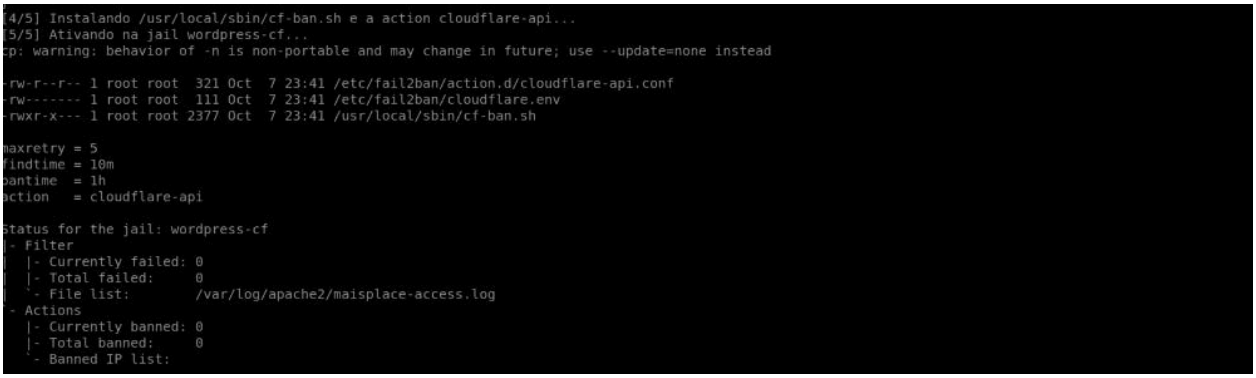


Figura 19 - Arquivo sigiloso do token (cloudflare.env) e inicialização da Action no Fail2ban.

O código da action em shell script chama a API, extraindo o Token através das variáveis de ambiente de forma segura, sem expor hard-coded na lógica.



Figura 20 - Código de integração provando que o token não vaza no script.

Prova da Cadeia Completa (Teste Real): Para validar, realizamos um teste final de ponta-a-ponta a partir de uma rede 4G. Ao bater o limite de erros estipulado pelo Fail2ban, perdemos acesso ao site imediatamente.

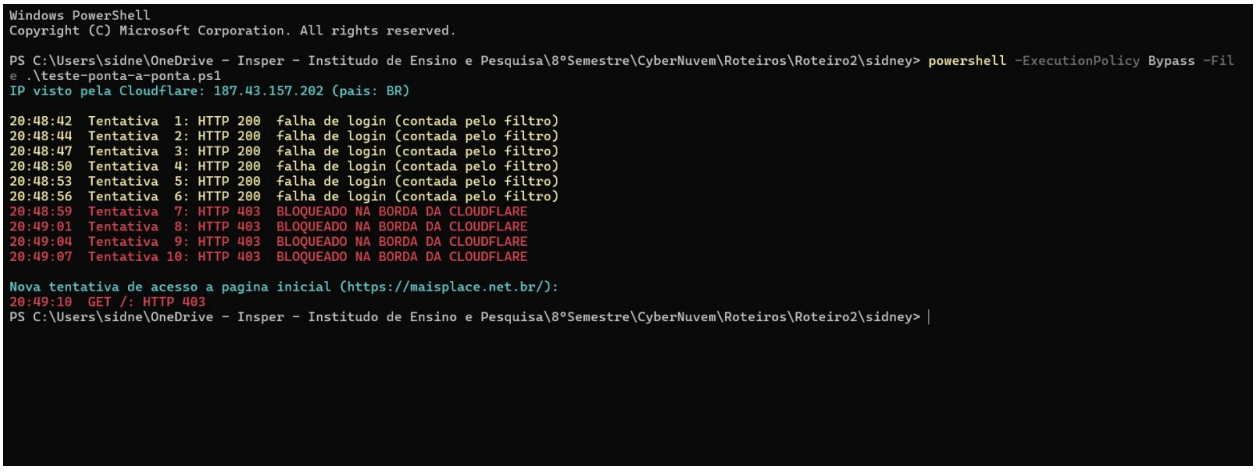


Figura 21 - Terminal do atacante tomando HTTP 403 Forbidden a partir da 6ª tentativa.

No lado do servidor, vemos o Fail2ban enviando os dados para a nossa action interagir com a Cloudflare.

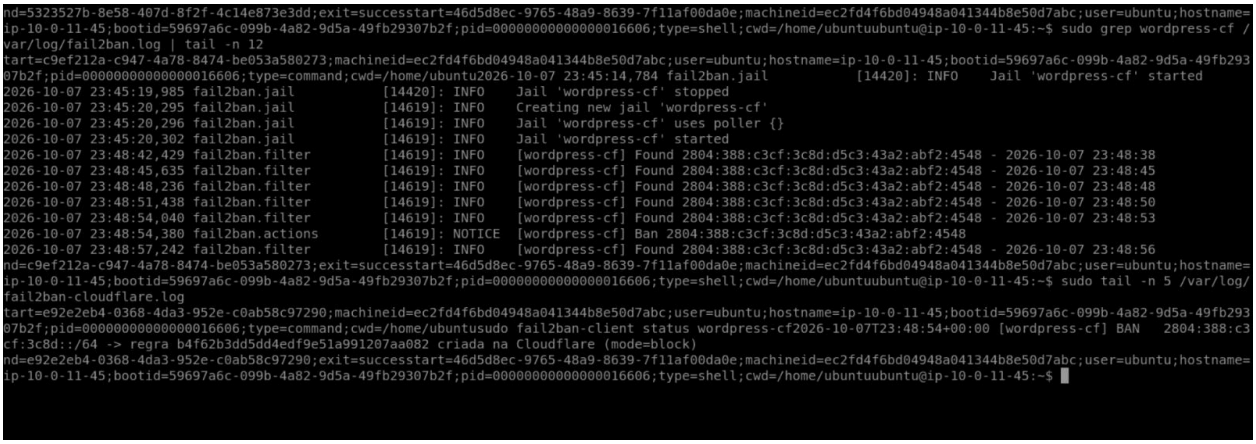


Figura 22 - Log do servidor apontando a criação do bloqueio pela API.

Checando o painel na nuvem da Cloudflare, vimos a mágica acontecer: a automação criou uma *IP Access Rule* bloqueando perfeitamente a sub-rede do atacante.

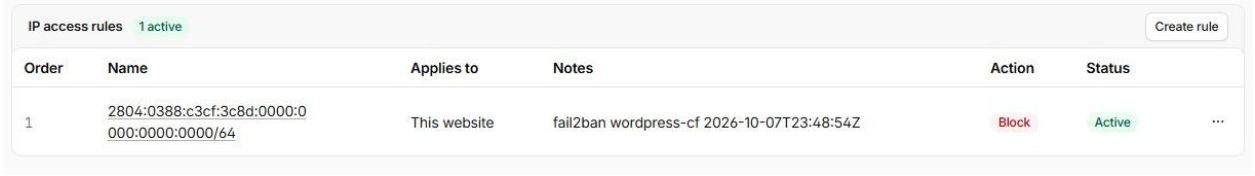


Figura 23 - Regra de bloqueio gerada por API e ativa na Cloudflare.

No módulo de segurança, notamos os ataques subsequentes caindo no Event Log e sendo negados na borda.

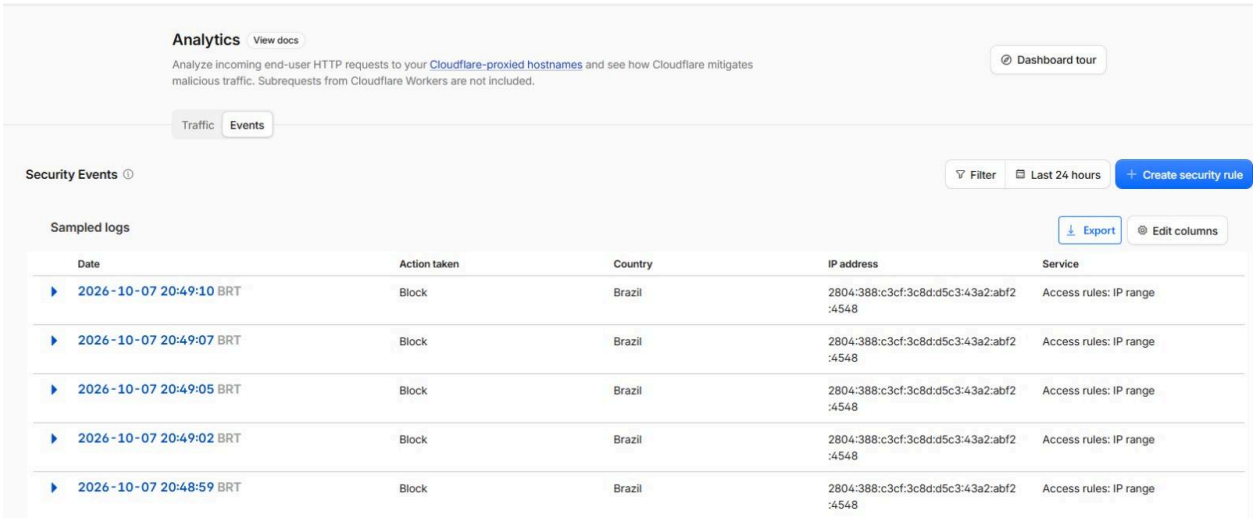


Figura 24 - Painel de eventos bloqueando os pacotes na origem.

Por fim, ao tentarmos navegar legitimamente pelo site usando o computador do atacante recém-banido, recebemos o Erro da Cloudflare provando que a automação funciona.

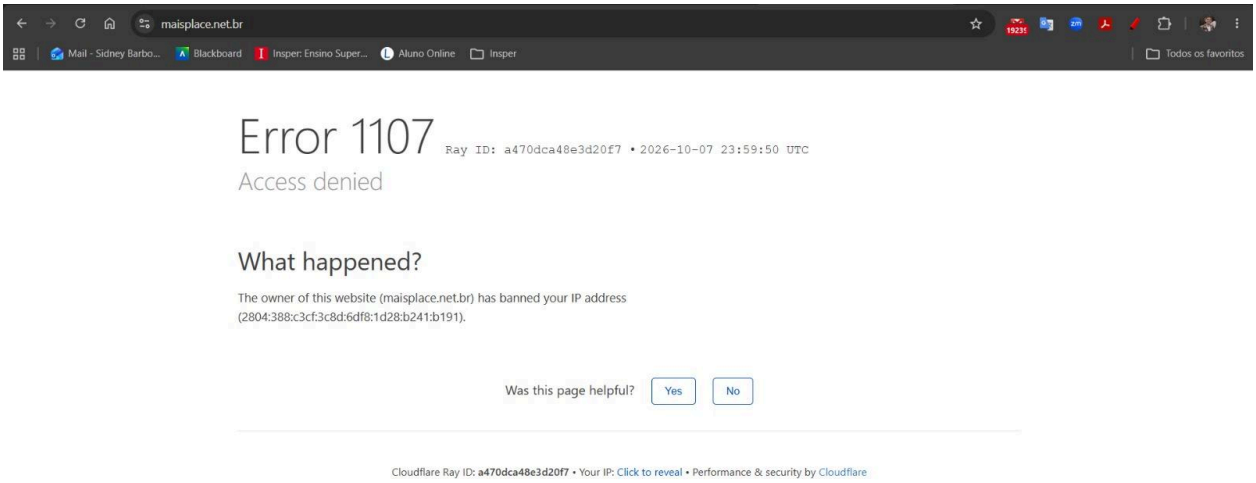


Figura 25 - Tela de "Erro 1107 - Access Denied", comprovando que o cliente está bloqueado.

Tarefa 3 – Restringindo o acesso ao servidor Web

O servidor Web está atrás do proxy reverso da Cloudflare. Se as portas 80/443 ficassem abertas para a Internet (0.0.0.0/0), qualquer pessoa mal-intencionada que descobrisse o IP do nosso servidor poderia acessá-lo diretamente, escapando do WAF e das regras automatizadas criadas na Tarefa 2. Para evitar esse contorno (*bypass*), limitamos o Security Group da AWS.

3.1 Configuração da Prefix List

Primeiro, buscamos as redes oficiais IPv4 e IPv6 da Cloudflare listadas publicamente.

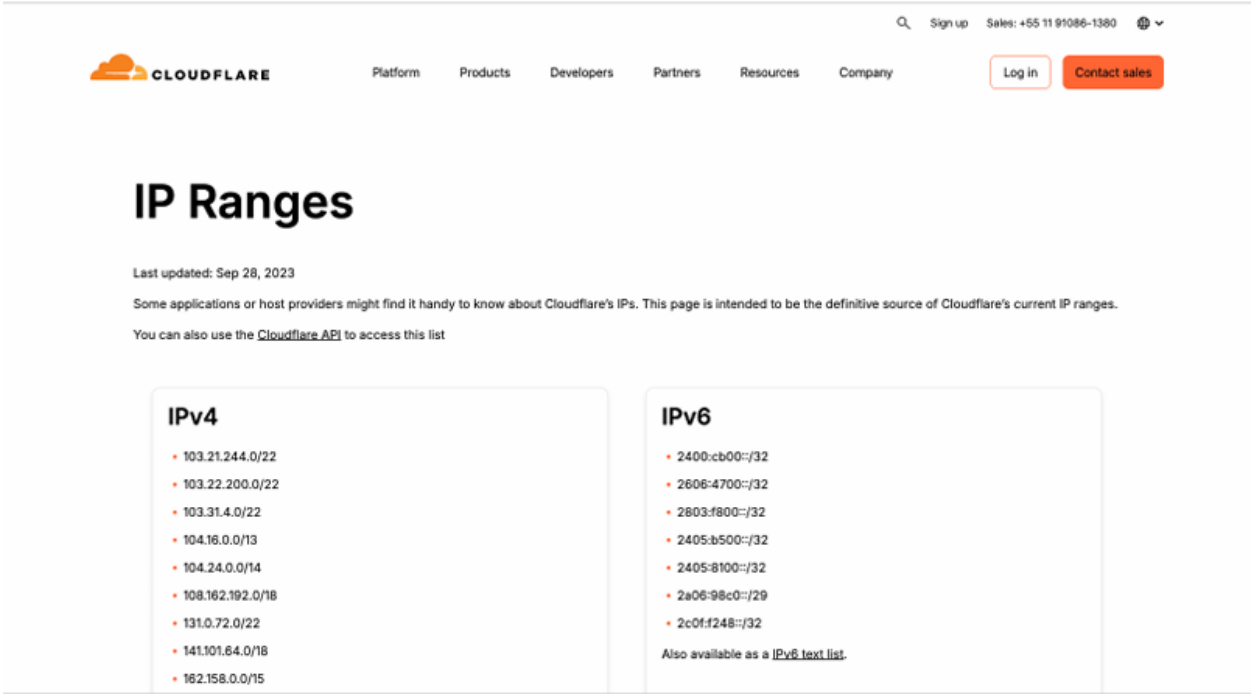


Figura 26 - Página oficial cloudflare.com/ips com os ranges.

Cadastramos todas essas redes numa *Managed Prefix List* da AWS.

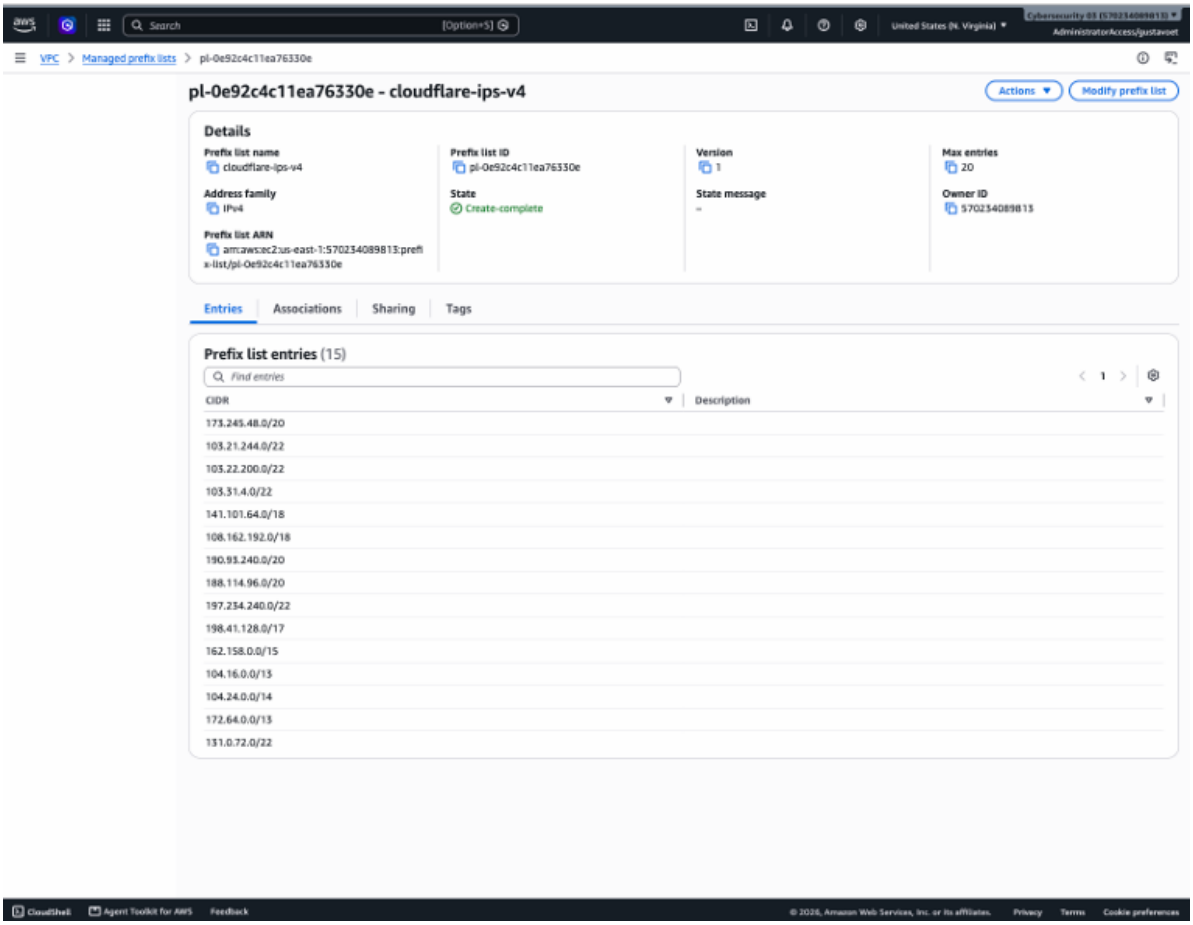


Figura 27 - Prefix list cloudflare-ips-v4 cadastrada na AWS contendo as origens oficiais.

O *Security Group* do Webserver foi atualizado. Agora, as portas HTTP (80) e

HTTPS (443) só dão permissão de entrada para requisições que venham dessa Prefix List da Cloudflare.

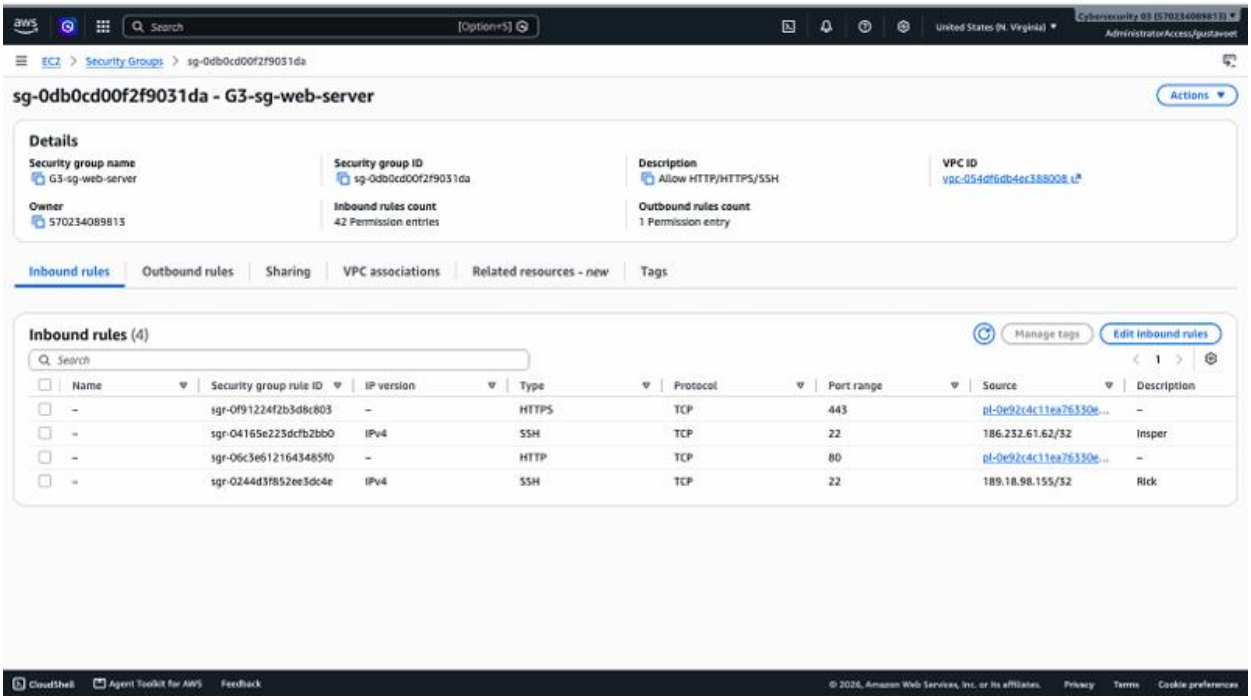


Figura 28 - Security Group do Webserver exibindo as regras de entrada rígidas.

3.2 Validação da Restrição

Para provar que o Bypass foi impedido, usamos uma máquina externa.

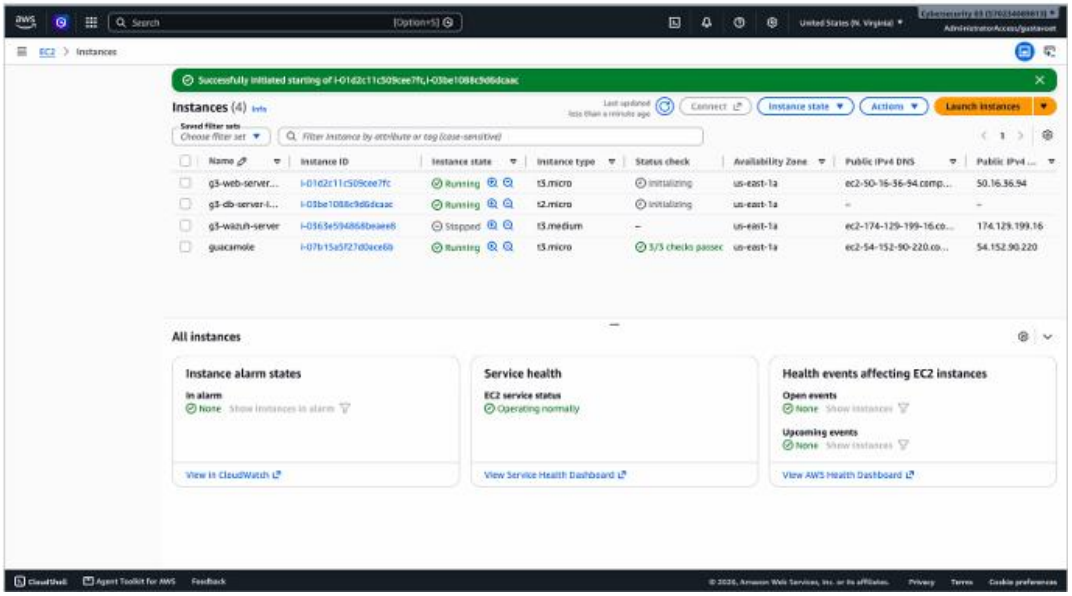


Figura 29 - Instâncias ativas.

Acessar o site pelo domínio (via DNS e roteamento Cloudflare) continuou funcionando perfeitamente.

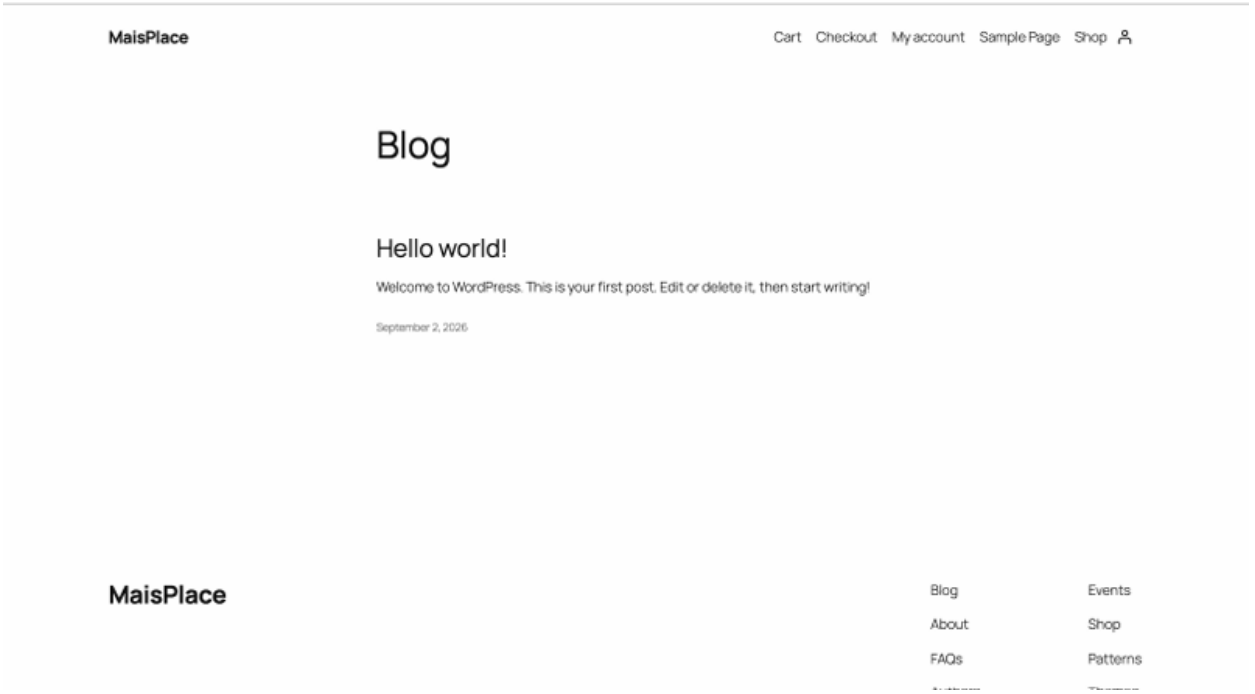


Figura 30 - Acesso realizado com sucesso através de maisplace.net.br.

Porém, tentar conversar diretamente com o servidor disparando curl para o IP público (50.16.36.94) falhou. O Security Group barrou tudo e a conexão sofreu *Time Out*.

```
$ bash tarefa3/valida-tarefa3.sh
Data: 06/10/2026 11:02:36 -03   Meu IP público: 186.232.61.65

[1] Acesso pelo domínio (via Cloudflare) - deve responder
    https://maisplace.net.br/  ->  HTTP 200 (conectou em 172.67.193.32)
    http://maisplace.net.br/   ->  HTTP 301 -> https://maisplace.net.br/

[2] Acesso direto ao IP público do webserver (50.16.36.94) - deve dar timeout
    curl: (28) Connection timed out after 10001 milliseconds
    https://50.16.36.94/  ->  HTTP 000
    curl: (28) Connection timed out after 10005 milliseconds
    http://50.16.36.94/  ->  HTTP 000

[3] Mesmo com o Host/SNI corretos, direto no IP - deve dar timeout
    curl: (28) Connection timed out after 10001 milliseconds
    HTTP 000
$
```

Figura 31 - Script testando conexão; acesso direto recebe Time Out.

Isso também foi comprovado pelo navegador, quando a tentativa de pular a Cloudflare acabou sem resposta.

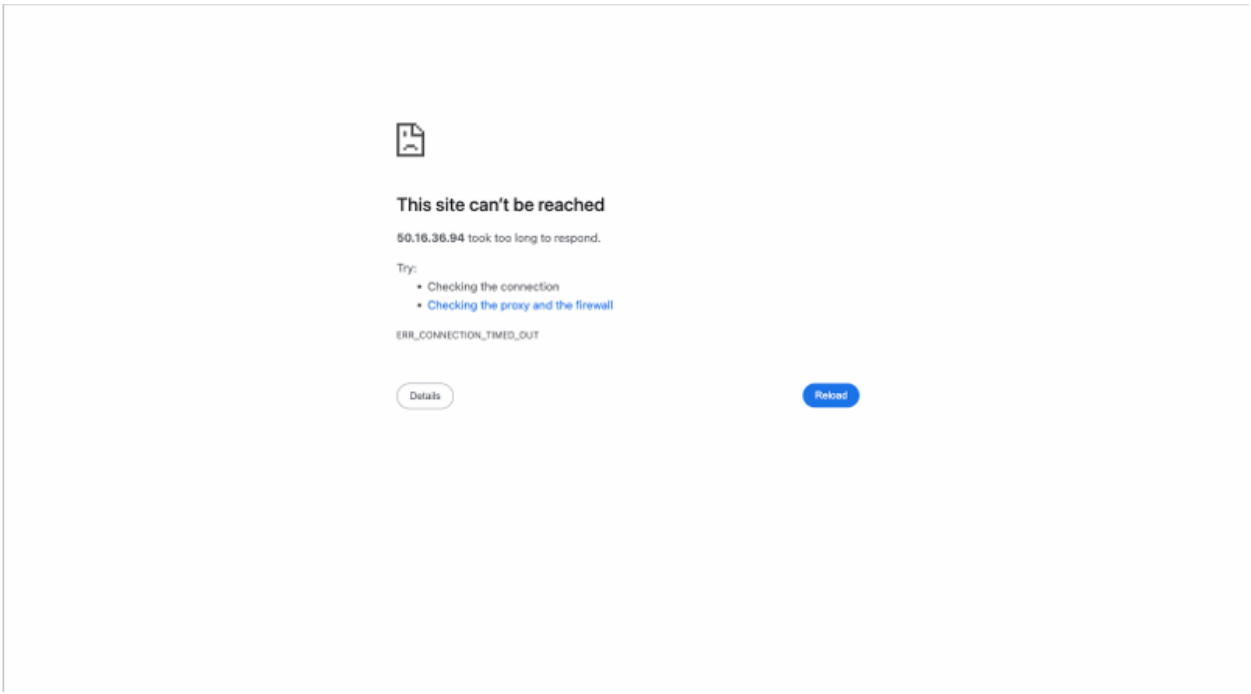


Figura 32 - Conexão negada (ERR_CONNECTION_TIMED_OUT) no navegador pelo IP direto.